



Prepared for
Aslan Tashtanov
Tony Lee
Sam Barani
Mysten Labs

Prepared by
Junghoon Cho
Sunwoo Hwang
Zellic

November 12, 2025

Deepbook Margin

Smart Contract Security Assessment

Placeholder text for the main body of the report.

Contents

About Zellic	4
1. Overview	4
1.1. Executive Summary	5
1.2. Goals of the Assessment	5
1.3. Non-goals and Limitations	5
1.4. Results	5
2. Introduction	6
2.1. About Deepbook Margin	7
2.2. Methodology	7
2.3. Scope	9
2.4. Project Overview	9
2.5. Project Timeline	10
3. Detailed Findings	10
3.1. Borrow share is incorrectly tracked	11
3.2. Bad debt with partial liquidation	13
3.3. Loss of accumulated referral fees until first manual claim	15
3.4. Fees accumulated to the default address cannot be claimed	16
3.5. The <code>margin_pool_id</code> is not cleared after being fully liquidated	18
3.6. Changing the referral causes all accumulated referral fees to be lost	20
3.7. Lack of config check	21
3.8. The protocol relies on a single price oracle	22

3.9.	Misleading comment in the EWMA state	24
4.	Discussion	25
4.1.	Lack of leverage check	26
4.2.	Test suite	26
5.	System Design	26
5.1.	Component: margin_pool	27
5.2.	Component: margin_manager	28
5.3.	Component: pool_proxy	30
6.	Assessment Results	33
6.1.	Disclaimer	34

About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the [#1 CTF \(competitive hacking\) team](#) worldwide in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io and follow [@zellic_io](#) on Twitter. If you are interested in partnering with Zellic, contact us at hello@zellic.io.



1. Overview

1.1. Executive Summary

Zellic conducted a security assessment for Mysten Labs from October 15th to November 12th, 2025. During this engagement, Zellic reviewed Deepbook Margin's code for security vulnerabilities, design issues, and general weaknesses in security posture.

1.2. Goals of the Assessment

In a security assessment, goals are framed in terms of questions that we wish to answer. These questions are agreed upon through close communication between Zellic and the client. In this assessment, we sought to answer the following questions:

- Could oracle failures or manipulation create systemic bad debt or prevent proper liquidation?
 - Does `liquidate()` correctly enforce penalty and reward distributions without allowing overliquidation?
 - Are there any vulnerabilities that could result in total asset drainage from the DeepBook or margin pools?
 - Can any function bypass collateral or ownership checks to transfer funds out of the pools?
 - Is there adequate validation for oracle price freshness (`max_age_secs`) and feed identifiers?
-

1.3. Non-goals and Limitations

We did not assess the following areas that were outside the scope of this engagement:

- Front-end components
- Infrastructure relating to the project
- Key custody

Due to the time-boxed nature of security assessments in general, there are limitations in the coverage an assessment can provide.

1.4. Results

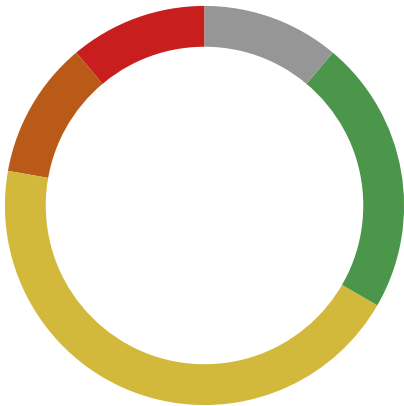
During our assessment on the scoped Deepbook Margin modules, we discovered nine findings. One critical issue was found. One was of high impact, four were of medium impact, two were of low

impact, and the remaining finding was informational in nature.

Additionally, Zellic recorded its notes and observations from the assessment for the benefit of Mysten Labs in the Discussion section ([4. 7](#)).

Breakdown of Finding Impacts

Impact Level	Count
Critical	1
High	1
Medium	4
Low	2
Informational	1



2. Introduction

2.1. About Deepbook Margin

Mysten Labs contributed the following description of Deepbook Margin:

DeepBook Margin enables margin trading directly on DeepBook, a fully on-chain central limit order book (CLOB) built on the Sui blockchain.

2.2. Methodology

During a security assessment, Zellic works through standard phases of security auditing, including both automated testing and manual review. These processes can vary significantly per engagement, but the majority of the time is spent on a thorough manual review of the entire scope.

Alongside a variety of tools and analyzers used on an as-needed basis, Zellic focuses primarily on the following classes of security and reliability issues:

Basic coding mistakes. Many critical vulnerabilities in the past have been caused by simple, surface-level mistakes that could have easily been caught ahead of time by code review. Depending on the engagement, we may also employ sophisticated analyzers such as model checkers, theorem provers, fuzzers, and so on as necessary. We also perform a cursory review of the code to familiarize ourselves with the modules.

Business logic errors. Business logic is the heart of any smart contract application. We examine the specifications and designs for inconsistencies, flaws, and weaknesses that create opportunities for abuse. For example, these include problems like unrealistic tokenomics or dangerous arbitrage opportunities. To the best of our abilities, time permitting, we also review the contract logic to ensure that the code implements the expected functionality as specified in the platform's design documents.

Integration risks. Several well-known exploits have not been the result of any bug within the contract itself; rather, they are an unintended consequence of the contract's interaction with the broader DeFi ecosystem. Time permitting, we review external interactions and summarize the associated risks: for example, flash loan attacks, oracle price manipulation, MEV/sandwich attacks, and so on.

Code maturity. We look for potential improvements in the codebase in general. We look for violations of industry best practices and guidelines and code quality standards. We also provide suggestions for possible optimizations, such as gas optimization, upgradability weaknesses, centralization risks, and so on.

For each finding, Zellic assigns it an impact rating based on its severity and likelihood. There is no hard-and-fast formula for calculating a finding's impact. Instead, we assign it on a case-by-case basis based on our judgment and experience. Both the severity and likelihood of an issue affect its impact. For instance, a highly severe issue's impact may be attenuated by a low likelihood. We assign the following impact ratings (ordered by importance): Critical, High, Medium, Low, and

Informational.

Zellic organizes its reports such that the most important findings come first in the document, rather than being strictly ordered on impact alone. Thus, we may sometimes emphasize an "Informational" finding higher than a "Low" finding. The key distinction is that although certain findings may have the same impact rating, their *importance* may differ. This varies based on various soft factors, like our clients' threat models, their business needs, and so on. We aim to provide useful and actionable advice to our partners considering their long-term goals, rather than a simple list of security issues at present.

Finally, Zellic provides a list of miscellaneous observations that do not have security impact or are not directly related to the scoped modules itself. These observations — found in the Discussion (4. ↗) section of the document — may include suggestions for improving the codebase, or general recommendations, but do not necessarily convey that we suggest a code change.

2.3. Scope

The engagement involved a review of the following targets:

Deepbook Margin Modules

Type	Move
Platform	Sui
Target	deepbookv3
Repository	https://github.com/MystenLabs/deepbookv3 ↗
Version	Only changes between 22c784b...82286e0
Programs	deepbook/** deepbook_margin/**

2.4. Project Overview

Zellic was contracted to perform a security assessment for a total of 5.2 person-weeks. The assessment was conducted by two consultants over the course of 4.2 calendar weeks.

Contact Information

The following project manager was associated with the engagement:

Chad McDonald
✈ Engagement Manager
chad@zellic.io ↗

The following consultants were engaged to conduct the assessment:

Junghoon Cho
✈ Engineer
junghoon@zellic.io ↗

Sunwoo Hwang
✈ Engineer
sunwoo@zellic.io ↗

2.5. Project Timeline

The key dates of the engagement are detailed below.

October 15, 2025	Kick-off call
-------------------------	---------------

October 15, 2025	Start of primary review period
-------------------------	--------------------------------

November 12, 2025	End of primary review period
--------------------------	------------------------------

3. Detailed Findings

3.1. Borrow share is incorrectly tracked

Target	margin_manager.move		
Category	Coding Mistakes	Severity	Critical
Likelihood	High	Impact	Critical

Description

The `borrow_base` and `borrow_quote` functions are used through the margin manager to borrow assets from a margin pool.

```
public fun borrow_base<BaseAsset, QuoteAsset>(
    [...]
) {
    registry.load_inner();
    self.validate_owner(ctx);
    [...]
    let (coin, total_borrow, total_shares) =
        base_margin_pool.borrow(loan_amount, clock, ctx);
    self.borrowed_base_shares = total_shares;

    public fun borrow_quote<BaseAsset, QuoteAsset>(
        [...]
    ) {
        [...]
        let (coin, total_borrow, total_shares) =
            quote_margin_pool.borrow(loan_amount, clock, ctx);
        self.borrowed_quote_shares = total_shares;
    }
}
```

These functions execute the actual borrow operation from the margin pool, receiving back the borrowed assets along with metadata about the total borrow amount and the number of shares of the pool. Then, the `borrowed_{base,quote}_shares` field of the margin manager is set to the `total_shares` value returned. However, `total_shares` indicates the global total of all users' debt shares, whereas `borrowed_{base,quote}_shares` tracks only those of an individual margin manager.

Impact

The accounting for loans from the margin pool breaks, causing the core logic of the related protocol to function incorrectly.

Recommendations

Modify the `borrow_base` and `borrow_quote` functions so that the `borrowed_{base,quote}_shares` field accumulates only the total debt shares of the corresponding margin manager.

Remediation

This issue has been acknowledged by Mysten Labs, and a fix was implemented in [PR #608](#). They intend to merge these changes into the production branch.

3.2. Bad debt with partial liquidation

Target	margin_manager.move		
Category	Business Logic	Severity	High
Likelihood	High	Impact	High

Description

In the `liquidate` function, the minimum repay amount is not enforced.

```
let repay_shares = if (risk_ratio < constants::float_scaling() && repay_amount
    == max_repay) {
    borrowed_shares
} else {
    math::mul(
        borrowed_shares,
        math::div(repay_amount, debt),
    )
let mut out_amount = math::mul(repay_amount,
    liquidation_reward_with_user_pool); // 97.087 * 1.05 = 101.941
```

When the `repay_amount` is small enough to be truncated in `math::div(repay_amount, debt)` with large debt, it can still get a profit from liquidation without decreasing shares.

Impact

This may cause bad debt, where the borrowed shares are not decreased while the margin manager's balance is reduced.

Bad debt can be caused by partial liquidation.

Recommendations

Enforce the minimum repay amount.

Remediation

This issue has been acknowledged by Mysten Labs, and fixes were implemented in the following PRs:

- [PR #655 ↗](#)
- [PR #659 ↗](#)

They intend to merge these changes into the production branch.

3.3. Loss of accumulated referral fees until first manual claim

Target	referral_fees.move		
Category	Business Logic	Severity	Medium
Likelihood	High	Impact	Medium

Description

In the current implementation, the `withdraw_referral_fees` function uses `referral_tracker.min_shares` to calculate the referral fees. However, `min_shares` is not updated in the `increase_shares` function when the referral is registered in the `supply` function located in `margin_pool`.

```
public(package) fun increase_shares(self: &mut ProtocolFees, referral:
    Option<ID>, shares: u64) {
    let referral_id =
        referral.destroy_with_default(margin_constants::default_referral());
    let referral_tracker = self.referrals.borrow_mut(referral_id);
    referral_tracker.current_shares = referral_tracker.current_shares + shares;
    self.total_shares = self.total_shares + shares;
}
```

Impact

Since `min_shares` is only updated in `withdraw_referral_fees`, until the referral manually calls `withdraw_referral_fees`, the referral would lose the fees accumulated up to that point.

Recommendations

Update `min_shares` in the `increase_shares` function when the referral is registered in the `supply` function.

Remediation

This issue has been acknowledged by Mysten Labs, and a fix was implemented in [PR #629](#). They intend to merge these changes into the production branch.

3.4. Fees accumulated to the default address cannot be claimed

Target	referral_fees.move		
Category	Business Logic	Severity	Medium
Likelihood	Medium	Impact	Medium

Description

The referral system is designed to incentivize users to bring liquidity suppliers to the margin pools. At its core, when someone supplies assets to a margin pool, they can optionally specify a referral address, and that referral earns a share of the interest fees generated by the pool.

```
public(package) fun calculate_and_claim(
    self: &mut ReferralFees,
    referral: &mut SupplyReferral,
    ctx: &TxContext,
): u64 {
    assert!(ctx.sender() == referral.owner, ENotOwner);
    [...]
}
```

Specifically, the system works through a shared object called `SupplyReferral` that anyone can mint. Suppliers can specify a referral address when depositing assets. Then, the referral-fee shares increase under the specified referral address, and the referral can later withdraw the accumulated fees through their `SupplyReferral` object. However, if the supplier does not specify a referral, the zero address is used as a default referral address. In this case, the referral fees become unclaimable, even by the admin.

Impact

All referral fees generated from suppliers who did not specify a referral become unclaimable, resulting in a loss of assets at the protocol level.

Recommendations

Modify the logic of the referral-fee system so that when a supplier does not specify a referral, either the protocol admin or the supplier themselves can claim the corresponding referral fees.

Remediation

This issue has been acknowledged by Mysten Labs, and a fix was implemented in [PR #628](#). They intend to merge these changes into the production branch.

3.5. The margin_pool_id is not cleared after being fully liquidated

Target	margin_manager.move		
Category	Coding Mistakes	Severity	Medium
Likelihood	Medium	Impact	Medium

Description

Users are prevented from simultaneously borrowing from both base assets and quote assets from margin pools, enforced by the can_borrow function.

```
// Helper function to determine if margin manager can borrow from a margin pool
fun can_borrow<BaseAsset, QuoteAsset, BorrowAsset>(
    self: &MarginManager<BaseAsset, QuoteAsset>,
    margin_pool: &MarginPool<BorrowAsset>,
): bool {
    let no_current_loan = self.margin_pool_id.is_none();

    self.margin_pool_id.contains(&margin_pool.id()) || no_current_loan
}
```

The can_borrow function checks whether the pool from which the user is attempting to borrow an asset matches the margin_pool_id field, which records the pool from which the user has already borrowed assets.

```
fun repay<BaseAsset, QuoteAsset, RepayAsset>(
    self: &mut MarginManager<BaseAsset, QuoteAsset>,
    margin_pool: &mut MarginPool<RepayAsset>,
    amount: Option<u64>,
    clock: &Clock,
    ctx: &mut TxContext,
): u64 {
    [...]
    if (self.borrowed_base_shares == 0 && self.borrowed_quote_shares == 0) {
        self.margin_pool_id = option::none();
    };
    [...]
}
```

In the repay function, once a user fully repays the borrowed assets, the margin_pool_id is cleared, allowing the user to borrow again from any pool.

However, the liquidation process does not include similar logic.

Impact

Even after a user is fully liquidated, their `margin_pool_id` remains set to the pool from which they previously borrowed. This prevents the user from borrowing assets from another pool postliquidation.

Recommendations

Modify the `liquidate` function so that when a user is fully liquidated, their `margin_pool_id` is cleared.

Remediation

This issue has been acknowledged by Mysten Labs, and a fix was implemented in [PR #633](#). They intend to merge these changes into the production branch.

3.6. Changing the referral causes all accumulated referral fees to be lost

Target	referral_fees.move		
Category	Business Logic	Severity	Medium
Likelihood	High	Impact	Medium

Description

In the supply function in margin_pool, when the referral is changed, the previous referral's share is set to zero.

```
let (total_user_supply_shares, previous_referral) = self
    .positions
    .increase_user_supply(supplier_cap_id, referral, supply_shares);

self.protocol_fees.decrease_shares(previous_referral, total_user_supply_shares
    - supply_shares);
self.protocol_fees.increase_shares(referral, total_user_supply_shares);
```

Impact

Since the withdraw_referral_fees function uses referral_tracker.min_shares to calculate referral fees, all referral fees accumulated before the referral change are lost.

Recommendations

Add a variable to track the accumulated referral fees when the share is changed.

Remediation

This issue has been acknowledged by Mysten Labs, and a fix was implemented in [PR #632](#). They intend to merge these changes into the production branch.

3.7. Lack of config check

Target	protocol_config.move		
Category	Coding Mistakes	Severity	Low
Likelihood	Medium	Impact	Low

Description

The `create_margin_pool` function accepts a `ProtocolConfig` as an argument. This struct is created in the `protocol_config` module, but it does not validate config values such as `optimal_utilization` and `max_utilization_rate`. These config values are directly related to risk management, so it is important to validate them.

Impact

While this is not a security issue, incorrect config values can pose risks to the protocol. In particular, incorrect `optimal_utilization` and `max_utilization_rate` values can increase the risk when borrowing assets, potentially leading to higher borrowing costs or insufficient liquidity management.

Recommendations

Add validation checks for each config value.

Remediation

This issue has been acknowledged by Mysten Labs, and a fix was implemented in [PR #660](#). They intend to merge these changes into the production branch.

3.8. The protocol relies on a single price oracle

Target	oracle.move		
Category	Business Logic	Severity	Low
Likelihood	Low	Impact	Low

Description

The protocol relies on oracle price feeds for several operations throughout its margin trading implementation. The oracle is used for the following:

- Converting user-deposited assets to USD value to determine borrowing capacity
- Computing the health factor of margin positions to determine if they are at risk
- Calculating the exact amount of assets to seize and debt to repay during liquidations
- Converting between different asset types when calculating total position value

As of the time of writing, the protocol exclusively uses Pyth Network as its sole oracle provider.

Impact

Relying on a single price oracle can introduce several potential risks. The following are examples of such risks:

- **Single point of failure.** Any disruption to Pyth Network's service — whether due to network outages, API changes, or infrastructure issues — would make the entire protocol inoperable.
- **Manipulation risks.** If Pyth Network's price feeds are compromised, manipulated, or report erroneous data due to bugs or external attacks, these incorrect prices would propagate throughout the protocol without any validation or cross-reference mechanism. This could enable unfair liquidations where users lose their collateral based on false price information or allow malicious actors to overborrow against inflated collateral valuations.

Recommendations

Implement a multioracle architecture that aggregates price data from multiple independent sources to eliminate single points of failure and enhance price accuracy.

Remediation

This issue has been acknowledged by Mysten Labs, and a fix was implemented in [PR #664](#). They intend to merge these changes into the production branch.

3.9. Misleading comment in the EWMA state

Target	ewma.state		
Category	Coding Mistakes	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

The `init_ewma_state` function returns an initialized EWMA state so that other packages can utilize it.

```

/// The Exponentially Weighted Moving Average (EWMA) state for DeepBook
[...]
/// The state is enabled by default and can be configured with different
    parameters.

public(package) fun init_ewma_state(ctx: &TxContext): EWMAState {
    let gas_price = ctx.gas_price() * constants::float_scaling();

    EWMAState {
        mean: gas_price,
        variance: 0,
        alpha: constants::default_ewma_alpha(),
        z_score_threshold: constants::default_z_score_threshold(),
        additional_taker_fee: constants::default_additional_taker_fee(),
        last_updated_timestamp: 0,
        enabled: false,
    }
}

```

By default, the function returns the state with the `enabled` field set to `false`, meaning the state is disabled. However, the comment in the module incorrectly states that the EWMA state is enabled by default.

Impact

The misleading comment may confuse users who refer to the smart contract and make it difficult to understand the intended behavior of the code. It can also have a negative impact on the maintainability of the code.

Recommendations

Update the comment or implementation to align with the protocol's design intent.

Remediation

This issue has been acknowledged by Mysten Labs, and a fix was implemented in [PR #644](#). They intend to merge these changes into the production branch.

4. Discussion

The purpose of this section is to document miscellaneous observations that we made during the assessment. These discussion notes are not necessarily security related and do not convey that we are suggesting a code change.

4.1. Lack of leverage check

Unlike `new_pool_config_with_leverage`, the `margin_registry::new_pool_config` function does not limit leverage. As a result, a `PoolConfig` created with `new_pool_config` can exceed the maximum leverage.

The Mysten Labs team replied with the following:

This is not a concern until we allow permissionless pool creation. Only authorized participants can create a pool and update the pool config currently. We want to keep the max leverage flexible without needing an upgrade.

4.2. Test suite

The DeepBook Margin implementation's test suite provides generally good test coverage across all components. As of the time of writing, the test suite ensures the protocol's robustness by testing the various entry points through which users can interact with each component. In addition, for the `margin_manager` and `margin_pool` modules, the tests cover not only the entry points but also the mathematical logic using several input values.

However, we have identified that tests for extreme values and some negative test cases are still missing. To maximize protocol safety, we recommend adding tests that cover all branches that can be visited in the implementation.

5. System Design

This provides a description of the high-level components of the system and how they interact, including details like a function's externally controllable inputs and how an attacker could leverage each input to cause harm or which invariants or constraints of the system are critical and must always be upheld.

Not all components in the audit scope may have been modeled. The absence of a component in this section does not necessarily suggest that it is safe.

5.1. Component: margin_pool

Description

The margin_pool is a lending protocol for margin trading, enabling liquidity providers to deposit assets and earn interest while margin traders borrow these assets for leveraged trading. It operates through a share-based accounting mechanism where supply shares represent ownership of deposited capital and borrow shares represent outstanding debt, efficiently distributing interest and fees across all participants without requiring individual account updates.

Interest accrues based on a two-slope interest-rate model determined by the utilization rate, with a portion of the generated interest carved out as fees and distributed to referrals (50%), pool maintainers (25%), and the protocol (25%).

Invariants

- The total supply must always be greater than or equal to the total borrow amount.
- The vault must hold enough tokens to satisfy the available liquidity, covering `total_supply - total_borrow`.
- As interest accrues, the value of each supply share increases over time.
- As interest accrues, the debt represented by each borrow share increases over time.
- Users can only withdraw supply shares they own through their `SupplierCap`.
- A `margin_manager` can only have debt in one margin pool at a time.
- Referral fees can only be claimed by the owner of the `SupplyReferral` object.

Test coverage

Cases covered

- Supplying collateral to the pool and withdrawing specific amounts
- Withdrawing all supplied collateral
- Multiple users supplying and withdrawing simultaneously
- Interest accrual over time and withdrawal with accumulated interest
- Supply-cap enforcement
- Max utilization-rate enforcement

- Minimum borrow-amount validation
- Insufficient liquidity checks
- Enabling/disabling deepbook pools for borrowing
- Liquidation with reward
- Liquidation with default
- Exact amount liquidation
- Zero-shares liquidation
- Partial liquidation

Cases not covered

- Changing referral after initial supply
- Withdrawing protocol fees by the MarginAdminCap holder
- Attempting to withdraw fees with invalid capabilities
- Attempting to repay more shares than borrowed
- Unauthorized users attempting to enable/disable deepbook pools without MarginPoolCap

Attack surface**Entry points**

- supply
- withdraw

Mitigations

- All entry points validate the sender is the MarginAdminCap holder.
- It is ensured that total supply does not exceed the configured supply cap.
- It is ensured that the vault balance increases atomically with state.
- Referral shares are properly tracked with the protocol_fees module.
- It is ensured that the withdrawal amount does not exceed the vault balance.
- It is ensured that only the shares owned by the specific SupplierCap can be withdrawn.
- Share calculations prevent overwithdrawal beyond the user's position.

5.2. Component: margin_manager**Description**

The margin_manager module is responsible for managing individual leveraged trading positions within the DeepBook margin trading. It provides users with the ability to borrow assets from margin pools to trade with leverage on DeepBook's order book, while maintaining proper risk management and collateralization requirements.

The module achieves this by wrapping a BalanceManager object along with deposit, withdraw, and trade capabilities, tracking borrowed shares for both base and quote assets, and continuously monitoring the risk ratio using oracle price feeds.

When users borrow, withdraw, or get liquidated, the manager enforces risk thresholds defined in the MarginRegistry, ensuring that positions can only borrow up to a maximum risk ratio and can be liquidated when they fall below the liquidation threshold. The MarginManager acts as a shared object owned by a specific user, allowing them to deposit collateral, borrow funds, execute trades through DeepBook, repay loans, and potentially face liquidation.

Invariants

- A MarginManager can only have an active loan in at most one margin pool at any time.
- After borrowing, $\text{risk_ratio} \geq \text{min_borrow_risk_ratio}$.
- After withdrawal, $\text{risk_ratio} \geq \text{min_withdraw_risk_ratio}$.
- For liquidation, $\text{risk_ratio} < \text{liquidation_risk_ratio}$.
- Only the owner can deposit, withdraw, borrow, repay, or trade.
- The MarginManager is permanently bound to its DeepBook pool set at creation.

Test coverage

Cases covered

- Margin-manager creation
- Deposits of base, quote, and DEEP assets
- Invalid deposits with incorrect asset types
- Withdrawals respecting risk-ratio limits
- Borrowing base and quote assets with risk-ratio validation
- Preventing simultaneous loans from multiple margin pools
- Full and partial repayments
- Liquidation
- Risk-ratio calculations with multiple assets and price changes
- Oracle price changes affecting positions
- Repayment rounding edge cases

Cases not covered

- Non-owner attempting operations
- Setting and unsetting referrals

Attack surface

Entry points

- `new`
- `new_with_initializer`
- `share`
- `set_referral`
- `unset_referral`
- `deposit`
- `withdraw`
- `borrow_base`
- `borrow_quote`
- `repay_base`
- `repay_quote`
- `liquidate`

Mitigations

- All owner-restricted operations validate `ctx.sender() == owner`.
- Risk ratio is enforced.
- It is ensured that users can only borrow from one margin pool at a time.
- Pool enablement is verified during creation.
- Borrowing is only allowed from margin pools that have explicitly whitelisted the DeepBook pool.
- Deposit-asset validation ensures only base, quote, or DEEP assets can be deposited.
- Liquidation validates that correct DeepBook pool and margin pool IDs match the manager's state.

5.3. Component: pool_proxy

Description

The `pool_proxy` module serves as an adapter layer between the DeepBook margin trading implementation and the core DeepBook order-book pools, enabling margin traders to interact with liquidity pools through their margin accounts. It exposes all essential trading operations including placing and canceling limit and market orders, modifying existing orders, and participating in pool governance through staking and voting mechanisms. The component works by performing necessary validation checks to ensure the margin manager is associated with the correct pool and that margin trading is enabled, then extracting trading credentials from the margin manager before delegating actual execution to the underlying pool functions.

Invariants

- A `MarginManager` can only interact with its designated DeepBook pool.
- Only pools explicitly enabled for margin trading in the registry can process regular trading operations.

- Reduce-only orders must decrease net exposure.
- DEEP tokens cannot be staked through a margin manager where either base or quote asset is DEEP.

Test coverage

Cases covered

- Placing limit/market orders
- Placing reduce-only limit/market orders
- Rejecting orders when using an incorrect pool
- Rejecting reduce-only orders that do not actually reduce position
- Rejecting reduce-only orders when quantity exceeds net debt
- Modifying orders
- Canceling orders
- Staking DEEP tokens
- Unstaking DEEP tokens
- Submitting governance proposals
- Voting on proposals
- Withdrawing settled amounts
- Claiming rebates

Cases not covered

- Non-owner attempting to place orders through another user's margin manager
- Placing orders with invalid quantities
- Modifying orders to invalid quantities
- Placing orders without sufficient collateral
- Voting on nonexistent proposals

Attack surface

Entry points

- `place_limit_order`
- `place_market_order`
- `place_reduce_only_limit_order`
- `place_reduce_only_market_order`
- `modify_order`
- `cancel_order`
- `cancel_orders`
- `cancel_all_orders`

- `withdraw_settled_amounts`
- `stake`
- `unstake`
- `submit_proposal`
- `vote`
- `claim_rebates`

Mitigations

- All functions verify that the margin manager's `deepbook_pool` matches the provided pool.
- Trade authorization is verified through `trade_proof` derived from the margin manager and transaction context.
- Regular trading functions ensure the pool is enabled for margin trading via the registry check.
- Reduce-only orders validate that the order actually reduces position by checking debt vs assets.
- Staking is prevented when the margin manager uses DEEP as a base or quote asset.
- Most parameter validation (balance sufficiency, order limits, proposal validity) is delegated to the underlying Pool module.

Components: `packages/deepbook/sources/state/ewma.move`, `packages/deepbook/sources/balance_manager.move`, and `packages/deepbook/sources/pool.move`

Description

The diff for `deepbook/*` includes the following changes.

Relating to the EWMA fee,

- an `EWMAState` in `state/ewma.move` was added to smooth gas prices with configurable `alpha`, `z_score_threshold`, and `additional_taker_fee`;
- during order creation, the pool updates EWMA then passes the current state to `state.process_create`; and
- inside `state.process_create`, the taker fee is computed from governance trade params and then adjusted by `ewma_state.apply_taker_penalty(taker_fee, ctx)`. This penalty is added to the taker fee.

Relating to the referral fee,

- after order processing and vault settlement, the pool charges an additional referral fee from the trader's `BalanceManager` and accrues it to `ReferralRewards`; and
- the referral fee is an extra deduction based on paid fees and the referral multiplier. Accrued rewards are later claimable by the referral owner.

Additionally, whitelisted pools do not charge maker/taker fees and are determined by governance proposal.

Invariants

- Preview paths like `get_quantity_out` and `get_order_deep_required` do not apply the EWMA penalty, so previews may be optimistic compared to execution.
- Each transaction's gas price is added to the EWMA state, which slightly increases the taker fee. High gas fees do not increase the EWMA state immediately; the increase is gradual.
- Referral claim requires referral ownership.

Test coverage

Cases covered

- EWMA update and taker penalty application under high-gas conditions
- Referral accrual and claim flows with DEEP/base/quote fee assets
- Maker/taker settlement with whitelisted pools

Cases not covered

- Preview paths like `get_quantity_out` and `get_order_deep_required` do not apply the EWMA penalty, so previews may be optimistic compared to execution.
- Multiple extreme gas-price scenarios are not covered.

Attack surface

Entry points

- `place_market_order`
- `place_limit_order`

Mitigations

- The gas price is added to the EWMA state, which slightly increases the taker fee. High gas fees do not increase the EWMA state immediately; the increase is gradual.

6. Assessment Results

During our assessment on the scoped Deepbook Margin modules, we discovered nine findings. One critical issue was found. One was of high impact, four were of medium impact, two were of low impact, and the remaining finding was informational in nature.

6.1. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.